

# Sonara: AI-Assisted Audio Plugin

Sonara

## Overview

Sonara is an AI-assisted audio plugin built using JUCE that allows users to generate and apply audio effects through text input. The project started out as a sophisticated audio plugin but evolved into a more educational & beginner music production tool through user feedback and iterative reframing.

## Keywords

**Digital Audio Workstation (DAW):** Software used to record, edit, and produce music (e.g., Ableton Live, Logic Pro X, FL Studio).

**Plugin:** A software extension that runs inside a DAW to apply effects to audio.

**JUCE:** A C++ framework used to build cross-platform audio plugins and real-time audio applications. <https://github.com/juce-framework/JUCE>

**Digital Signal Processing (DSP):** Algorithms used to modify audio signals (e.g., filtering, compression, reverb).

**Equalizer (EQ):** A tool that adjusts the loudness of specific frequency ranges (e.g., boosting bass or reducing harsh high frequencies).

**Frequency Response:** A visualization of how much energy an audio signal has at different frequencies (used to validate EQ behavior).

## Problem

Modern digital audio workstations (DAWs) and plugins are powerful but have a steep learning curve. Beginners, even with a musical background, struggle to understand audio signal processing concepts (e.g., EQ, compression) and translate creative ideas into technical parameter adjustments. At the same time, existing AI music tools focus heavily on song generation, rather than manipulation and learning.

## Initial Approach

We started by building Sonara as a traditional audio plugin tool using JUCE that was meant to act as a copilot to music production. It was focused on real-time audio processing, core DSP effects (e.g., EQ, compression), and plugin integration into DAWs. To accelerate the development to include multiple effects, we wrote boilerplate DSP code manually to understand JUCE architecture. Then, we used a code generation tool, Cursor, to expand DSP modules.

## AI-Assisted Development and Validation

Instead of hand-writing every DSP component, we used an LLM to generate code for new audio effects. However, we needed a clean way to trust the code generated.

To solve this problem, we created a validation pipeline for new effects that were added. First was a quantitative validation. I plotted the output of the original vs. the transformed audio based on the effect. For example, for an Equalizer, I plotted the frequency response (Figure 1); for reverb, I plotted a spectrogram.

These plots validated against the expected behavior. For example, when the user asked the tool to “make brighter and increase bass,” the tool followed the expected behavior as shown below. The orange curve displays the boost in low end (below  $10^2$  Hz) and the boost in the high end (just under  $10^4$  Hz).

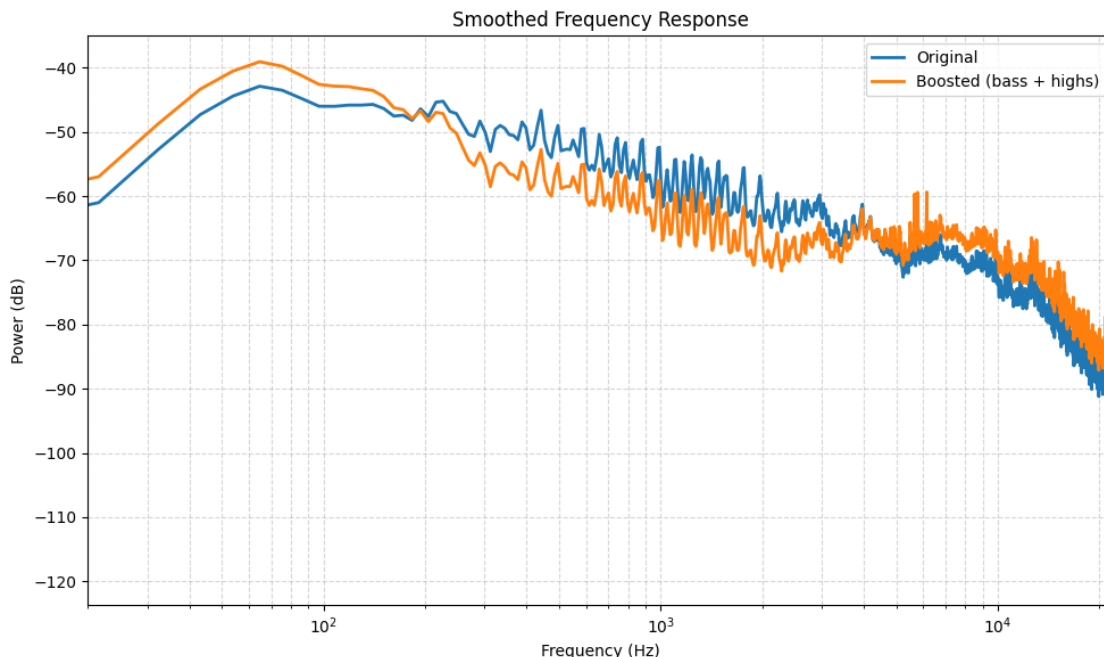


Figure 1: Example frequency response of low and high end boost EQ

Next, we validated using perceptual tests. We evaluated whether the output matched intended audio transformations and made sure there were no artifacts. Using the hybrid approach allowed us to confirm the AI-generated code and safely incorporate it into production.

## Pivot to EdTech Platform

After building the initial plugin, we spoke with potential users (student musicians, beginner producers). We quickly discovered that users didn’t just want tools but also wanted to learn what the tools were doing. This led to a reframing of Sonara as an educational audio platform, not just a plugin.

With the new direction, adjustments were accompanied with explanations of transformations, so users could map their text input with the technical adjustments. With this focus on learning by doing, we were able to fit beginner user needs and differentiate from existing tools.

## Challenges & Tradeoffs Summary

### 1. Real-Time Constraints

Since audio plugins require low latency, we realized that the LLM inference cannot run in the audio thread. We decoupled any LLM calls from real-time processing.

### 2. Reliability of Generated Code

LLM outputs are not guaranteed to be correct, so we used a validation layer with both

quantitative and perceptual tests.

### 3. **Product Positioning**

Through user interviews, we made a pivot to sit at the intersection of an impactful, educational, and creative AI tool.

### **Impact**

Sonara is a working audio plugin system built to optimize mixing workflows, lower the barrier to entry to music production, and teach the fundamentals of mixing effects. We used a novel workflow with LLM-generated digital signal processing, ultimately winning 2nd place at Georgia Tech's Junior Design Startup Competition.

### **Links**

If you would like to see Sonara in action, check out this early [Demo](#) (The product has since been iterated).

[Download Sonara](#)